A GENETIC ALGORITHM FOR JAZZ TENOR SAXOPHONE MELODIES

Carlos Henrique Brito Malta Leão
Universidade Federal de Minas Gerais (UFMG)
chbmleao@ufmg.br

ABSTRACT

This paper presents a genetic algorithm (GA) that composes a jazz tenor saxophone melody over a ii-V-I progression in C major (Dm7 | G7 | Cmaj7 | Cmaj7), played in a loop by a piano. Each melody is an array of integers with one gene per eighth note, where each gene is a MIDI pitch, a rest or a hold. The fitness combines five rules taken from jazz theory and from the Weimar Jazz Database: breathing, step size, chord tones on the beat, chromatic approach notes and guide tones at chord changes. We ran the GA with mutation rates of 0.01, 0.05 and 0.2, with three seeds each. The rate 0.01 reaches a best fitness of 0.999 on all seeds, while 0.2 stalls between 0.64 and 0.70. In an informal listening test, the melodies of the last generation sounded better than those of the first, but some notes still sounded out of place, which shows that following the rules does not guarantee good music.

1. INTRODUCTION

Jazz improvisation is spontaneous composition: the musician creates a melody in real time while following the harmony and reacting to the other players. It requires knowledge of harmony and rhythm, and it is also a way for the player to express feelings, moods and intentions.

Despite this freedom, improvisers follow rules. They target chord tones on strong beats, move mostly by small steps, approach notes chromatically and pause to breathe between phrases. Because these rules can be measured, they can be written as a score for a melody, which makes improvisation a natural problem for genetic algorithms (GAs), as shown by early systems such as GenJam [1] and by later work on GA composition [2].

Genetic algorithms are stochastic search and optimization methods first proposed by Holland [3], inspired by Darwin's theory of natural selection [4]. A GA keeps a population of individuals (candidate solutions), each encoded as a genome, a string of genes. A fitness function scores how well each individual solves the problem. At each generation, the GA selects parents with a probability that grows with their fitness and creates new individuals with crossover, which combines parts of two parents, and mutation, which randomly changes some genes. Over many generations, the population moves toward better solutions.

This paper presents a creative GA that improvises a jazz tenor saxophone melody over a chord progression played in a loop by a piano. The model combines a modified representation scheme with several fitness objectives, so melodies are generated automatically, without human feedback. Since evaluating such subjective work is difficult, the fitness relies on jazz music theory rules. Section 2 reviews related work, Section 3 describes the method, Section 4 presents the results, and Section 5 discusses them, with the limitations and future work.

2. RELATED WORK

Horner and Goldberg [5] published the first use of a genetic algorithm (GA) for music composition. Since then, several directions of GA application to melody composition have emerged. One notable example is GenJam [6], which uses an interactive genetic algorithm, in other words, a human-in-the-loop fitness evaluation. The system evolves two populations of musical elements (measures and phrases), with notes mapped to the scale of the current chord. The fitness of each individual comes from a human mentor, who gives real-time feedback by typing "g" for good or "b" for bad. This approach exposes the "fitness bottleneck": human evaluation limits the speed and scalability of the evolutionary process. In contrast, our work replaces the human mentor with jazz theory rules, so the fitness is evaluated automatically.

Besides interactive approaches, another line of research computes the fitness automatically from general music theory rules. Ozcan and Ercal [7] present AMUSE, a GA-based system that generates improvised melodies over a given harmonic context. Each gene is a scale degree, where 0 is a rest and the maximum value is a hold. Its fitness function combines ten objectives based on music theory, including rules for chord tones, relationships between notes, directions of notes, the beginning and ending notes of the melody, and the proportion of rests. In a Turing test, listeners had to tell apart solos generated by the GA from solos written by an amateur musician, and 52% of them gave the wrong answer, which suggests that the generated solos are comparable to an amateur's. Our work builds on this approach, using a similar rest/hold encoding and a fitness that combines several objectives.

Matić [8] also uses an automatic fitness, to compose short melodies of four bars. The genome follows the same rest/hold encoding, with one integer per eighth note, where 0 is a rest, the values from 1 to n are pitches relative to a reference note, and $n + 1$ is a hold. The fitness is minimized and computed bar by bar as a weighted sum of the

quality of each interval, the mean and variance of the intervals compared with reference values, and a penalty for notes outside the key. Since the melodies are short, the GA uses no crossover, only three mutations: moving a note by an octave, changing one note, and swapping two neighboring notes. The author reports that the GA tends to produce many rests, because rests avoid bad intervals. In contrast, our work keeps crossover, at bar lines, and rests come from an explicit breathing rule instead of being a side effect of avoiding bad intervals.

Besides these general techniques, Wiggins et al. [9] argue that the results of a GA depend heavily on the domain-specific knowledge in its operators and fitness function. Their jazz solo generator represents a solo as a list of (scale degree, duration) genes over a chord progression, and its fitness checks eight features, such as illegal jumps, long notes, speed, and pattern matching. They found that too much weight on pattern matching led to repetitive and boring solos. They also note that their systems have neither a clear plan nor intention, and that large-scale structure cannot be expected from a GA with little domain knowledge, a limitation that our work shares.

3. METHODOLOGY

Like AMUSE [7], our system uses a genetic algorithm (GA) to generate an improvised melody over a given harmony. Here, the melody is a jazz tenor saxophone line over a fixed ii-V-I progression in C major, Dm7 | G7 | Cmaj7 | Cmaj7, played in a loop. Since the chord of each bar is known, every rule can be checked bar by bar. We chose the tenor saxophone because the Weimar Jazz Database has mostly tenor solos (158 tenor against 80 alto), so the statistics we borrow from it fit this instrument [10].

3.1 Representation

With the problem scoped, we need a representation of a melody that the GA can manipulate. A melody is a genome, an array of integers with one gene per eighth note, so its length depends on the number of bars, which is a parameter. Each integer is either a MIDI pitch from 44 to 76 (the tenor saxophone range in concert pitch), a rest (43), or a hold (77) that sustains the previous note, the same rest and hold idea used in [7, 8]. The eighth note is the shortest duration and longer notes are built from holds, so there are no triplets or sixteenth notes.

Unlike AMUSE, whose genes are scale degrees, our genes are MIDI pitches, so the GA can produce notes outside the scale, such as the chromatic approach notes that are important in jazz improvisation. When a genome is decoded, consecutive equal pitches are separate notes, since only a hold extends a note. Notes are also split at bar lines, so each note belongs to a single chord, and a hold at the very start of the melody is treated as a rest.

3.2 Fitness Function

The fitness of a melody combines five rules. Each rule i gives a score s_i in $[0, 1]$, where 1 is best, and has a weight w_i that sets how much the rule counts in the final fitness. The fitness F is the weighted mean of the scores (Equation 1). The weights are a parameter of the GA, and in this paper all of them are equal to 1, chosen by hand, which is a limitation of this work.

$$F = \frac{\sum_{i=1}^{5} w_i \, s_i}{\sum_{i=1}^{5} w_i} \tag{1}$$

- *Breathing.* A saxophone player has to stop to breathe, so phrases cannot be too long. We measure the longest run of eighths played without a rest. The score is 1 if this run is at most 16 eighths (two bars), and it decreases linearly above that. The limit of 16 eighths is our own choice.
- *Step size.* Jazz solos move mostly by small intervals. Ignoring rests and repeated pitches, the score is the mean of two parts: how close the mean interval is to 2.7 semitones, $\max(0, 1 - |\bar{d} - 2.7|/2.7)$, and how close the share of leaps larger than 7 semitones is to 2%, $1 - \max(0, p - 0.02)/0.98$. Both reference values come from the Weimar Jazz Database [10].
- *Chord tones.* Improvisers usually play chord tones on the beat, as in AMUSE [7]. The score is the fraction of beats (even eighths) whose sounding note belongs to the chord of that bar, and beats with a rest are not counted.
- *Chromatic approach.* A note outside the C major scale is correct only if it starts off the beat and the next note is one half step away, as in a bebop chromatic approach. The score is the fraction of outside notes that are correct, and 1 if there are none.
- *Guide tones.* At each chord change, the first note of the new bar should be the 3rd or the 7th of the new chord, which gives a smooth voice leading between chords [11]. A rest at a chord change counts as a miss.

3.3 Genetic Operators

The GA evolves a population of 100 melodies with the steps below, in this order.

- *Initialization.* Each melody of the first population is a random walk. It starts at a random pitch, and each eighth is a rest with probability 0.2, a hold with probability 0.1, or else the pitch moves by at most 3 semitones.
- *Elitism.* The 2 best individuals are copied to the next generation unchanged, so the best fitness never decreases.
- *Selection.* Parents are chosen by roulette wheel, with a probability proportional to their fitness.
- *Crossover.* One-point crossover at a random bar line, which gives two children. Cutting at a bar line keeps each bar intact, so its notes stay aligned with its chord and guide tones.

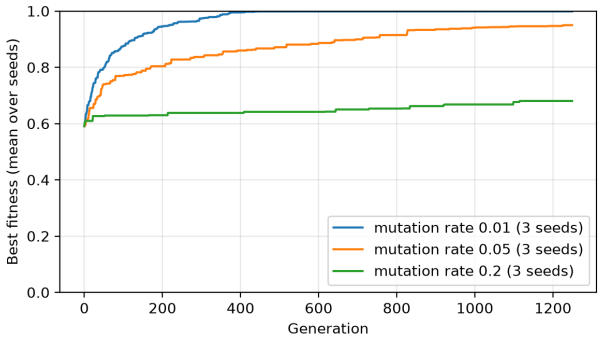


Figure 1. Best fitness per generation for each mutation rate, averaged over 3 seeds.

- *Mutation.* Each eighth of each child mutates with probability 0.01. A mutated eighth becomes a rest (probability 0.2), a hold (0.1), or else its pitch moves by up to ± 2 semitones. A rest or hold that mutates into a pitch gets a random pitch instead.
- *Stopping criterion.* The GA stops after a fixed number of generations (1250).

The best melody is rendered as MIDI, with the tenor saxophone over a piano that plays one voicing per bar, at 170 BPM, the median tempo in the Weimar Jazz Database [10], with a swing ratio of 1.3 [12].

4. RESULTS

To study the effect of the mutation rate, we ran the GA with three mutation rates, 0.01, 0.05 and 0.2, each with three random seeds (1, 2 and 3), for a total of 9 runs. All other parameters keep the default values of Section 3, so each run evolves a population of 100 melodies of 12 bars for 1250 generations. Each run saves the best melody of generations 0, 311, 624, 936 and 1249 as MIDI and audio files. The code, the MIDI files, the audio of the first and last generation of each run and the script that reproduces the experiment are available at `https://github.com/Chbmleao/jazz-sax-ga`.

Figure 1 compares the best fitness of the three mutation rates. With the same seed, all runs start from the same initial population, with a best fitness close to 0.59.

With mutation rate 0.01, the best fitness grows quickly: it passes 0.95 between generations 186 and 340 and 0.99 by generation 374, and it ends at 0.999 on all three seeds. In the run with seed 1, the mean fitness of the population rises from 0.48 to about 0.8 in the first 300 generations and then stays between 0.76 and 0.86 until the end, while the best fitness already reaches 0.995 by generation 400. In the final melodies, every rule scores 1.0 except the step size, which scores 0.99. With 0.05, the GA improves more slowly and ends between 0.906 and 0.986, and only two of the three seeds pass 0.95, after generation 800. With 0.2, the best fitness stalls between 0.64 and 0.70. Its weakest rule is the chromatic approach, which ends between 0.22 and 0.39, while the breathing rule scores 1.0 for every rate and seed. In all runs, the mean fitness of the population stays well below the best one, and it drops as the mutation rate grows: 0.82 at 0.01, 0.58 at 0.05 and 0.39 at 0.2.

Since the fitness measures how well a melody follows the rules, and not how good it sounds, we also listened to the generated melodies. For each run, the author compared the best melody of the first generation with the best melody of the last one (generation 1249). In most runs, the melody of the last generation sounded musically better. The melodies of the first generation sounded random, with many notes that did not seem to belong to the song, which made them uncomfortable to hear, and their melodic lines did not move in a way that made sense. In the last generation, the lines made a little more sense. However, some notes still sounded out of place. The chromatic approach rule tries to imitate the notes outside the scale that jazz musicians use, but in our melodies these notes did not sound good and did not convey the feeling of real jazz. This was an informal test, with a single listener who knew which melody came from which generation, so it is only a first indication that the rules lead to better melodies.

5. DISCUSSION AND FUTURE WORK

The results show that a low mutation rate works best for this problem. Since the mutation is applied to each eighth, a rate of 0.2 changes about one eighth in five in every child, so good bars are destroyed faster than selection can keep them. This also explains the low mean fitness of the population at 0.2. The chromatic approach rule suffers the most, because a correct approach note depends on two notes in specific positions, which a high mutation rate easily breaks.

A fitness close to 1.0, however, does not mean that the melody is good music. The final melodies with mutation rate 0.01 score 1.0 on the chromatic approach rule, but some of their notes still sounded out of place. Each rule checks the notes locally, so a melody can follow all of them and still sound wrong as a whole. This agrees with Wiggins et al. [9], who note that their systems have neither a clear plan nor intention, and that large-scale structure cannot be expected from a GA with little domain knowledge.

This work also has limitations. The progression and the key are fixed (a ii-V-I in C major), and the melodies only use eighth notes, with no triplets or sixteenths. The rule weights are all equal and were chosen by hand. Every saxophone note is also played with the same MIDI velocity, so the melodies sound robotic, while in real jazz the intensity of each note varies a lot, and this is one of the ways the soloist expresses feelings through the music.

As future work, the first step is a blind listening test: play the best melody of the first and of the last generation to several listeners, in random order and without saying which is which, and ask which one they prefer. This follows the Turing test of AMUSE [7], where listeners had to tell the solos of the GA apart from the solos of an amateur musician. Other extensions are to vary the velocity of each note, to support other progressions, keys and rhythms, and to tune the rule weights instead of choosing them by hand.

6. REFERENCES

[1] J. A. Biles, "GenJam: A genetic algorithm for generating jazz solos," in *Proc. of the Int. Computer Music Conf. (ICMC)*, Aarhus, Denmark, 1994, pp. 131–137.

[2] G. Papadopoulos and G. Wiggins, "A genetic algorithm for the generation of jazz melodies," in *Proc. of STeP'98*, Jyväskylä, Finland, 1998.

[3] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press, 1975.

[4] C. Darwin, *On the Origin of Species by Means of Natural Selection*. London: John Murray, 1859.

[5] A. Horner and D. E. Goldberg, "Genetic algorithms and computer-assisted music composition," in *Proc. of the Int. Computer Music Conf. (ICMC)*, 1991, pp. 479–482.

[6] J. A. Biles, "GenJam: An interactive genetic algorithm jazz improviser," Popular version of paper 4pMU1, 134th Meeting of the Acoustical Society of America, San Diego, CA, 1997.

[7] E. Özcan and T. Erçal, "A genetic algorithm for generating improvised music," in *Artificial Evolution (EA 2007)*, ser. Lecture Notes in Computer Science, vol. 4926. Springer, 2008, pp. 266–277.

[8] D. Matić, "A genetic algorithm for composing music," *Yugoslav Journal of Operations Research*, vol. 20, no. 1, pp. 157–177, 2010.

[9] G. Wiggins, G. Papadopoulos, S. Phon-Amnuaisuk, and A. Tuson, "Evolutionary methods for musical composition," Department of Artificial Intelligence, University of Edinburgh, DAI Research Paper 882, 1998.

[10] M. Pfleiderer, K. Frieler, J. Abeßer, W.-G. Zaddach, and B. Burkhart, Eds., *Inside the Jazzomat: New Perspectives for Jazz Research*. Mainz, Germany: Schott Campus, 2017.

[11] S. R. Smither, "Guide-tone space," *Music Theory Online*, vol. 25, no. 2, 2019.

[12] C. Corcoran and K. Frieler, "Playing it straight: Analyzing jazz soloists' swing eighth-note distributions with the Weimar Jazz Database," *Music Perception*, vol. 38, no. 4, pp. 372–385, 2021.

[13] Y.-W. Wen and C.-K. Ting, "Composing bossa nova by evolutionary computation," in *Proc. of the Int. Joint Conf. on Neural Networks (IJCNN)*, 2020, pp. 1–8.

[14] M. Towsey, A. Brown, S. Wright, and J. Diederich, "Towards melodic extension using genetic algorithms," *Educational Technology & Society*, vol. 4, no. 2, 2001.

[15] J. D. Fernández and F. Vico, "AI methods in algorithmic composition: A comprehensive survey," *Journal of Artificial Intelligence Research*, vol. 48, pp. 513–582, 2013.